

Содержание

1	Анализ прямой задачи	2
1.1	Импорт необходимых библиотек	2
1.2	Исследование зависимости коэффициента отражения от показателя преломления	3
1.3	Карта коэффициента отражения	5
1.4	Карта функции невязки	6
1.5	Исследование использования нескольких углов падения	7
1.5.1	Влияние количества углов	9
1.5.2	Влияние шума	10
1.5.3	Дополнительное исследование влияния шума на относительную погрешность	12
1.6	Восстановление показателя преломления при известной толщине слоя	15
2	Численные методы решения обратной задачи	18

direct_task

4 июля 2026 г.

1. Анализ прямой задачи

В формулы подставляется и варьируется величина волнового числа $k = \omega \sqrt{\mu_0 \mu \epsilon_0 \epsilon}$ и толщины слоя h .

Известно, что

$$c = \frac{1}{\sqrt{\mu_0 \epsilon_0}}$$

, тогда получаем, что

$$k = \frac{\omega}{c} \sqrt{\mu \epsilon}$$

, где

ω — круговая частота

c — скорость света

μ, ϵ — относительная магнитная и диэлектрическая проницаемость среды

Тогда $k_0 = \frac{\omega}{c} = \frac{2\pi}{\lambda_0}$ - волновое число вакуума, а $n = \sqrt{\mu \epsilon}$ - показатель преломления.

Для исследования некорректности обратной задачи зафиксируем длину волны λ_0 в коде и будем вычислять остальные величины как:

$$k_0 = \frac{2\pi}{\lambda_0}$$

$$\tilde{h} = k_0 h$$

$$\tilde{k} = \frac{k}{k_0} = n_i$$

То есть $k = n_i k_0$, а $\tilde{h} = k_0 h$ - безразмерные величины

1.1. Импорт необходимых библиотек

```
[24]: import numpy as np
import matplotlib.pyplot as plt
from physics import *
```

```
[25]: np.random.seed(21)
```

1.2. Исследование зависимости коэффициента отражения от показателя преломления

```
[26]: theta0 = np.deg2rad(30)

lambda0 = 2 * np.pi # Длина волны в метрах
k0 = 2 * np.pi / lambda0

n1 = 1.0 # воздух
n2 = 1.0 # подложка

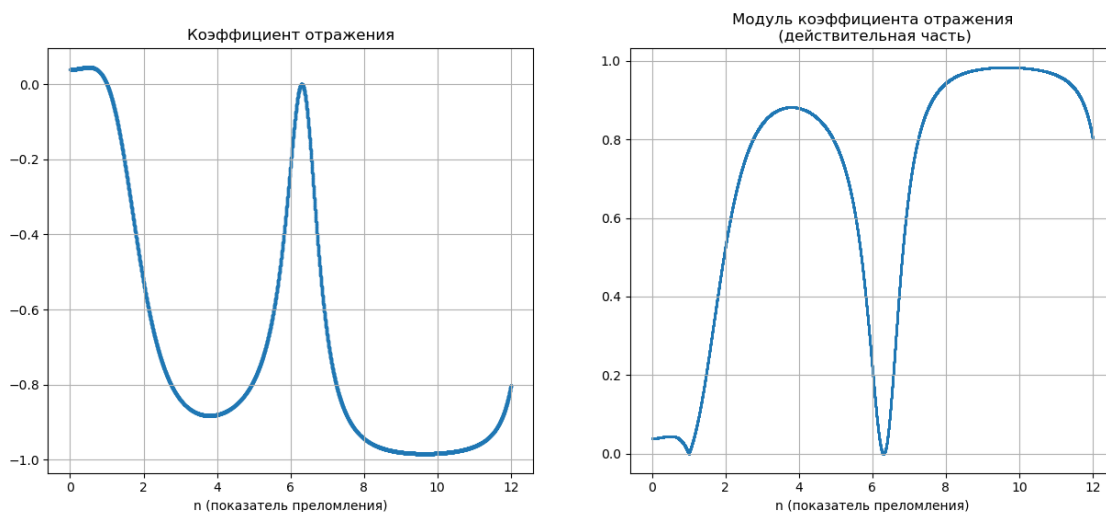
h_base = 0.5 # толщина слоя в метрах
theta0 = np.deg2rad(30) # Угол в радианах

[27]: n_vals = np.linspace(0.0, 12.0, 10000) # Показатели преломления
      r_vals = [r_total(np.array([n1, n, n2]) * k0, h_base, theta0) for n
      in n_vals] # Посчитанные значения коэффициентов отражения

[28]: fig, [ax1, ax2] = plt.subplots(1, 2, figsize=(15, 6))

ax1.scatter(n_vals, np.real(r_vals), s=2)
ax1.set_title('Коэффициент отражения')
ax1.set_xlabel('n (показатель преломления)')
ax1.grid()

ax2.scatter(n_vals, np.abs(np.real(r_vals))), s=1)
ax2.set_title('Модуль коэффициента отражения\n(действительная часть)')
ax2.set_xlabel('n (показатель преломления)')
ax2.grid()
```

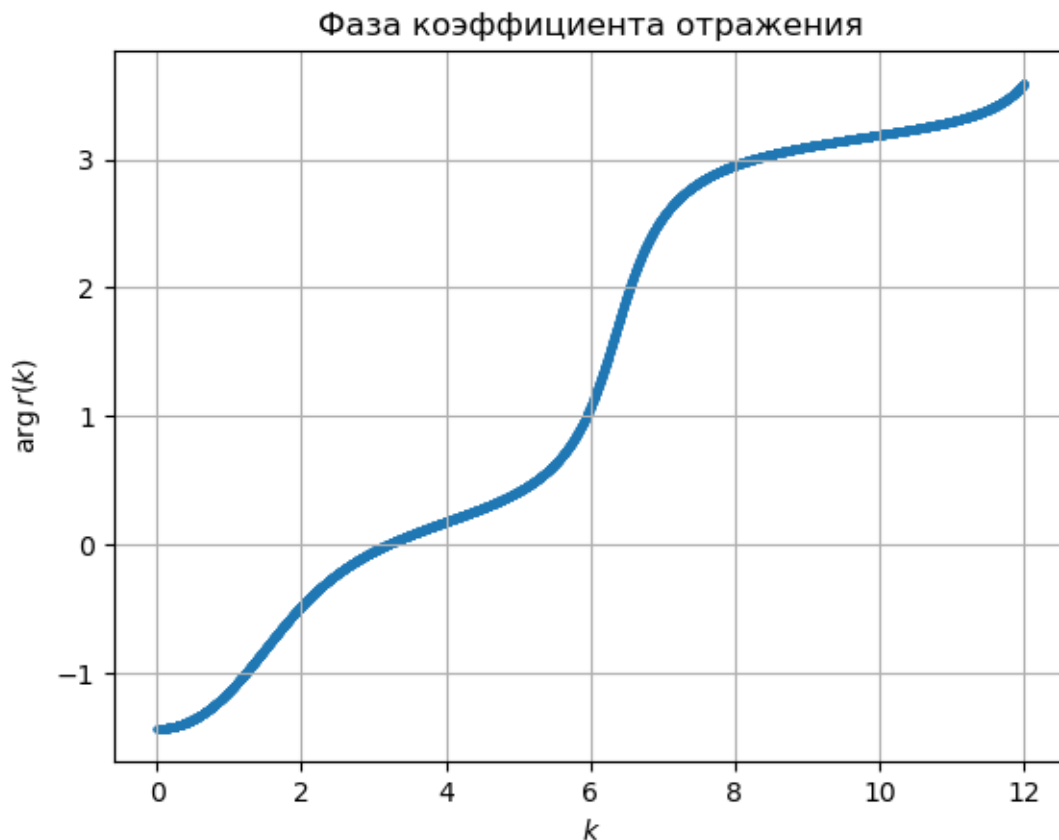


На графиках наблюдаются быстрые осцилляции коэффициента отражения как функции показателя преломления. Это означает, что небольшие изменения параметров среды могут приводить к существенным изменениям отраженного сигнала. Следовательно, при наличии шума в измерениях восстановление параметров может оказаться неустойчивым.

```
[29]: print("Максимум |r| =", np.max(np.abs(r_vals)))
```

Максимум $|r|$ = 0.9839617700984582

```
[30]: plt.figure()
plt.scatter(n_vals, np.unwrap(np.angle(r_vals), period=np.pi), s=3)
plt.xlabel(r"$k$")
plt.ylabel(r"$\arg r(k)$")
plt.title("Фаза коэффициента отражения")
plt.grid()
plt.show()
```

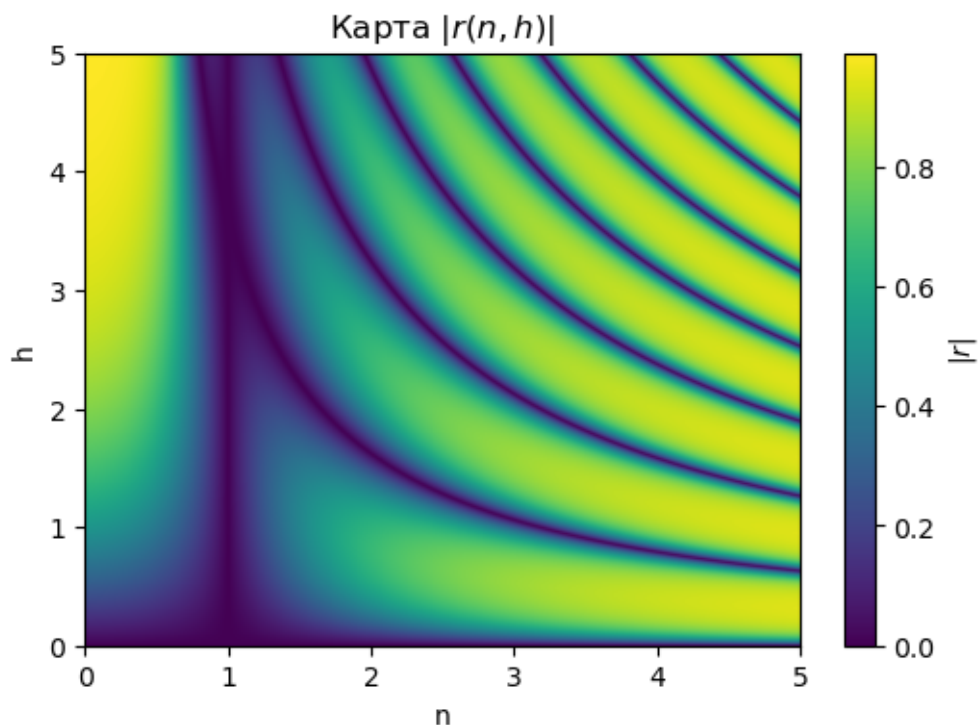


1.3. Карта коэффициента отражения

```
[31]: h_vals = np.linspace(0, 5, 1500)
      n_vals = np.linspace(0, 5, 1500)

      R = r_map(n_vals, h_vals, n1, n2, k0, theta0)

      plt.figure(figsize=(6, 4))
      plt.imshow(
          np.abs(R),
          origin="lower",
          extent=[n_vals.min(), n_vals.max(), h_vals.min(), h_vals.max()],
          aspect="auto"
      )
      plt.colorbar(label=r"$|r|$")
      plt.xlabel("n")
      plt.ylabel("h")
      plt.title(r"Карта $|r(n,h)|$")
      plt.show()
```



На этом графике наличие вытянутых долин может говорить о сильной корреляции между n, h , то есть изменение одного параметра может компенсироваться изменением второго, при этом давая близкие значения r .

1.4. Карта функции невязки

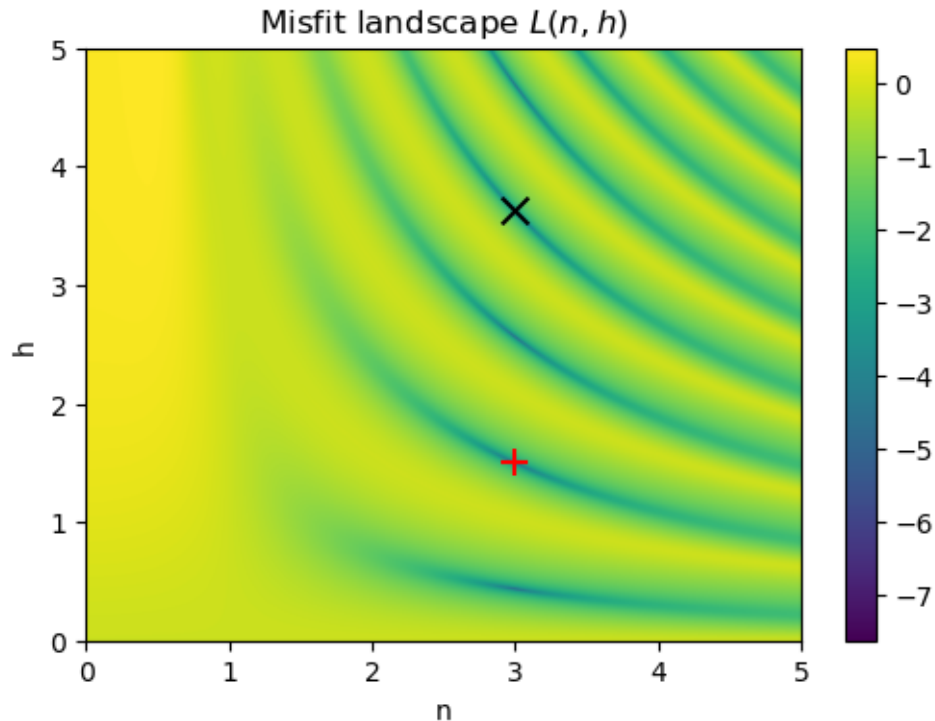
```
[32]: n_true = 3
      h_true = 1.5

      r_true = r_total(
          np.array([n1, n_true, n2]) * k0,
          h_true,
          theta0
      )

      R = r_map(n_vals, h_vals, n1, n2, k0, theta0)

      loss = np.abs(R - r_true) ** 2
      # loss = np.angle(R - r_true)**2

      plt.figure(figsize=(6, 4))
      plt.imshow(
          np.log10(loss),
          origin="lower",
          extent=[n_vals.min(), n_vals.max(), h_vals.min(), h_vals.max()],
          aspect="auto"
      )
      plt.colorbar()
      plt.xlabel("n")
      plt.ylabel("h")
      plt.title(r"Misfit landscape  $L(n,h)$ ")
      plt.scatter(
          n_true,
          h_true,
          c='red',
          s=100,
          marker='+'
      )
      idx = np.unravel_index(np.argmin(loss), loss.shape)
      plt.scatter(
          n_vals[idx[1]],
          h_vals[idx[0]],
          c='black',
          s=100,
          marker='x'
      )
      plt.show()
```



На карте невязки наблюдаются вытянутые долины локальных минимумов. Это означает, что одному и тому же измеренному коэффициенту отражения могут соответствовать различные пары параметров (n, h) . Следовательно, обратная задача неоднозначна при использовании данных только для одного угла падения.

1.5. Исследование использования нескольких углов падения

```
[33]: def test_multi_on_thetas(thetas, n_true, h_true, n_vals, h_vals, *,
    ↪ noise=False, percent_of_noise=0.05, graphs=False, seed=None):
    rng = np.random.default_rng(seed=seed)

    loss = np.zeros((len(n_vals), len(h_vals)), dtype=float)

    mins = []
    areas = []

    for theta in thetas:
        theta = np.deg2rad(theta)

        R_theta = r_map(n_vals, h_vals, n1, n2, k0, theta)

        r_true = r_total(
            np.array([n1, n_true, n2]) * k0,
```

```

        h_true,
        theta
    )

    if noise:
        eps_re = rng.uniform(-percent_of_noise, percent_of_noise)
        eps_im = rng.uniform(-percent_of_noise, percent_of_noise)
        r_true_noisy = r_true * (1 + eps_re + 1j*eps_im)

        loss += np.abs(R_theta - r_true_noisy)**2
    else:
        loss += np.abs(R_theta - r_true)**2

    idx = np.unravel_index(np.argmin(loss), loss.shape)

    delta = 1e-3
    area = np.sum(loss < loss.min() + delta)

    areas.append(area)
    mins.append(loss.min())

    if graphs:
        plt.figure(figsize=(6, 4))
        plt.imshow(
            np.log10(loss),
            origin="lower",
            extent=[n_vals.min(), n_vals.max(), h_vals.min(), h_vals.
↵max()],
            aspect="auto"
        )
        plt.colorbar()
        plt.xlabel("n")
        plt.ylabel("h")
        plt.title(r"Misfit landscape  $L(n,h)$ ")
        plt.scatter(
            n_true,
            h_true,
            c='red',
            s=100,
            marker='+'
        )
    idx = np.unravel_index(np.argmin(loss), loss.shape)
    plt.scatter(
        n_vals[idx[1]],
        h_vals[idx[0]],
        c='green',
        s=100,

```

```

        marker='x'
    )
    plt.show()

    deviation = (
        np.abs(
            np.array([
                n_vals[idx[1]],
                h_vals[idx[0]]
            ]) -
            np.array([
                n_true,
                h_true
            ])
        )
        /
        np.array([
            n_true,
            h_true
        ])
        * 100
    )

    return areas, deviation

```

1.5.1. Влияние количества углов

```

[34]: # thetas = [10, 30, 45, 60, 85]
      thetas = np.linspace(10, 89, 10)
      n_true = 4
      h_true = 2

      noise_values = np.linspace(0, 0.1, 20)
      h_vals = np.linspace(0, 5, 750)
      n_vals = np.linspace(0, 5, 750)

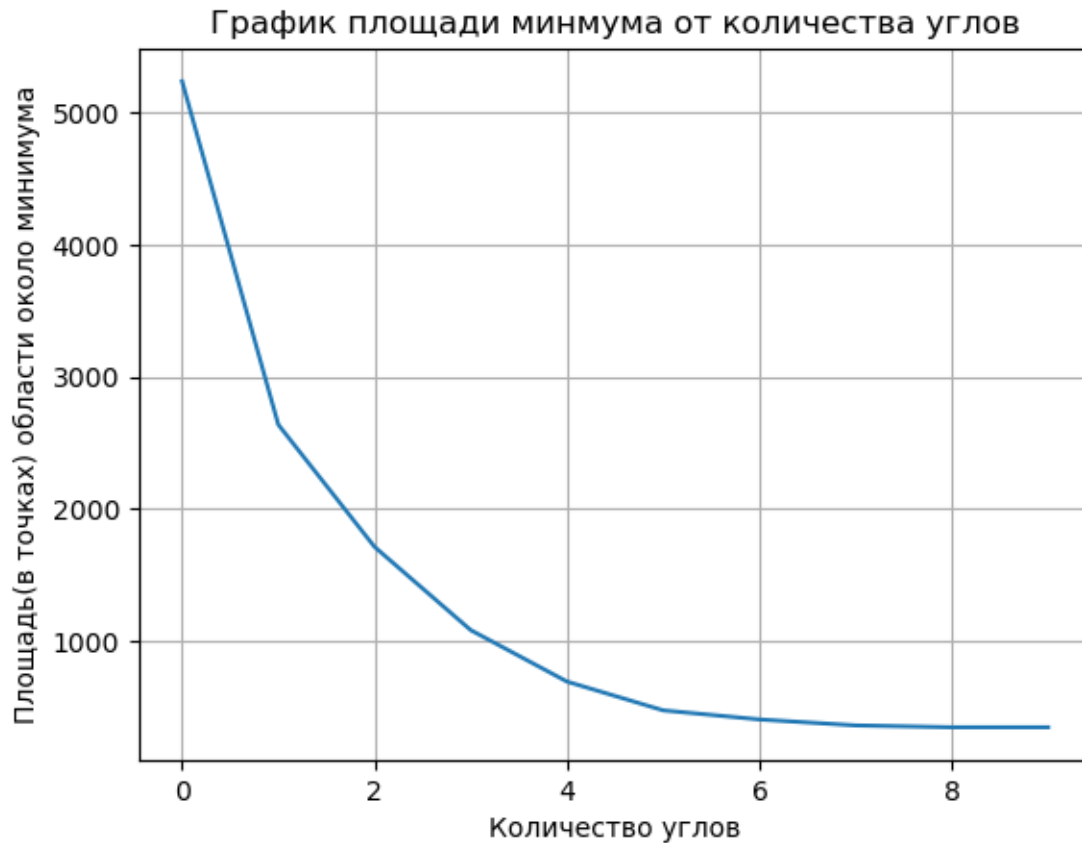
      area, deviation = test_multi_on_thetas(thetas, n_true, h_true,
      ↪ n_vals, h_vals, graphs=False, noise=False)

```

```

[35]: plt.plot(area)
      plt.title("График площади минмума от количества углов")
      plt.xlabel("Количество углов")
      plt.ylabel("Площадь(в точках) области около минимума")
      plt.grid(True)

```



1.5.2. Влияние шума

```
[71]: thetas = [10, 30, 45, 60, 85]
      # thetas = np.linspace(10, 89, 10)
      n_true = 4
      h_true = 2

      noise_values = np.linspace(0, 0.1, 20)
      h_vals = np.linspace(0, 5, 500)
      n_vals = np.linspace(0, 5, 500)

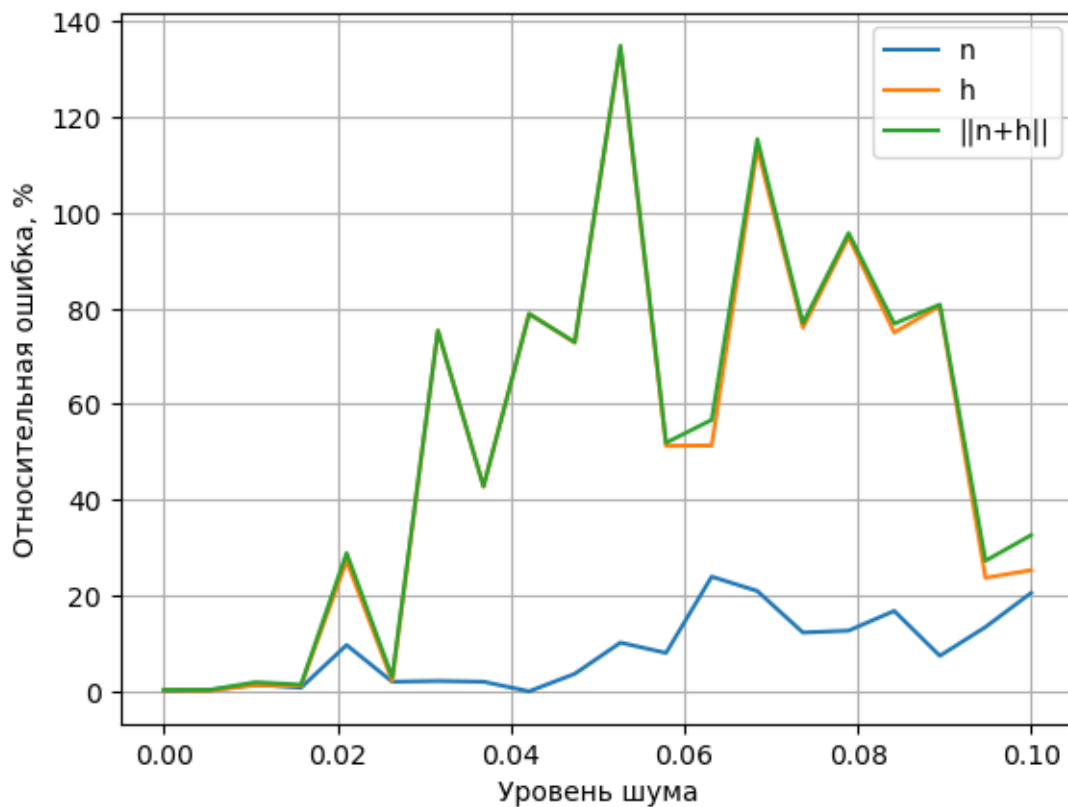
      deviations = []
      areas = []

      for noise_value in noise_values:
          area, deviation = test_multi_on_thetas(thetas, n_true, h_true,
          ↪ n_vals, h_vals, graphs=False, noise=True,
          ↪ percent_of_noise=noise_value)
          deviations.append(deviation)
```

```
areas.append(area)
```

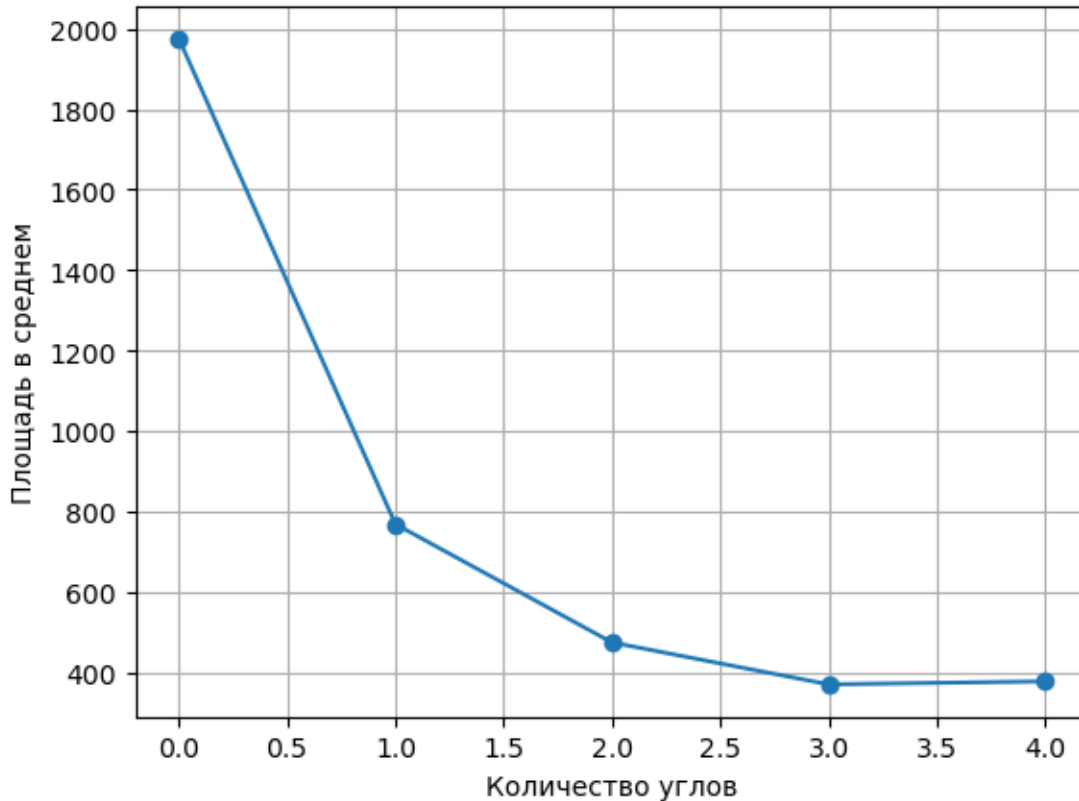
```
deviations = np.array(deviations)  
areas = np.array(areas)
```

```
[72]: plt.plot(noise_values, deviations[:,0], label="n")  
plt.plot(noise_values, deviations[:,1], label="h")  
plt.plot(noise_values, np.sqrt(deviations[:,1]**2 + deviations[:,0]**2), label="||n+h||")  
plt.xlabel("Уровень шума")  
plt.ylabel("Относительная ошибка, %")  
plt.legend()  
plt.grid()
```



На графике относительной погрешности можно видеть, что при увеличении уровня шума, величина ошибки восстановления параметров в среднем растет. То есть задача восстановления (n , h) чувствительна к шуму. Однако видно, что растет погрешность некоторыми скачками, что стоит отдельно изучить, но линия нижней тенденции в среднем растет.

```
[38]: plt.plot(np.average(areas, axis=0), marker='o')
plt.xlabel("Количество углов")
plt.ylabel("Площадь в среднем")
plt.grid()
```



На этом графике можно наблюдать, что в среднем площадь области минимума начинает уменьшаться с увеличением количества измерений на разных углах.

Анализируя построенные графики, можно сделать вывод о том, что хоть области минимумов невязки уменьшаются по мере увеличения измерений, однако при достаточно большом шуме эта область минимума может оказаться далеко от истинных значений, что говорит о неустойчивости задачи и ее неединственности.

1.5.3. Дополнительное исследование влияния шума на относительную погрешность

В функцию `test_multi_on_thetas()` была добавлена возможность фиксации сивда генератора псевдослучайных чисел.

Чтобы исключить влияние различных реализаций случайного шума, было принято решение использовать фиксированную последовательность псевдослучайных чисел для каждого уровня шума (путем установки `seed`). Таким образом, при

изменении уровня шума менялась только его амплитуда, а не реализация. Для получения статистически устойчивых результатов эксперимент затем повторялся для множества различных значений `seed`, после чего вычислялась средняя ошибка восстановления.

```
[64]: thetas = [10, 30, 45, 60, 85]
      # thetas = np.linspace(10, 89, 10)
      n_true = 4
      h_true = 2

      noise_values = np.linspace(0, 0.1, 20)
      h_vals = np.linspace(0, 5, 500)
      n_vals = np.linspace(0, 5, 500)

      N_EXPERIMENTS = 10

      mean_deviation = []
      std_deviation = []

      for noise in noise_values:

          errors_n = []
          errors_h = []

          for seed in range(N_EXPERIMENTS):

              _, deviation = test_multi_on_thetas(
                  thetas,
                  n_true,
                  h_true,
                  n_vals,
                  h_vals,
                  noise=True,
                  percent_of_noise=noise,
                  seed=seed
              )

              errors_n.append(deviation[0])
              errors_h.append(deviation[1])

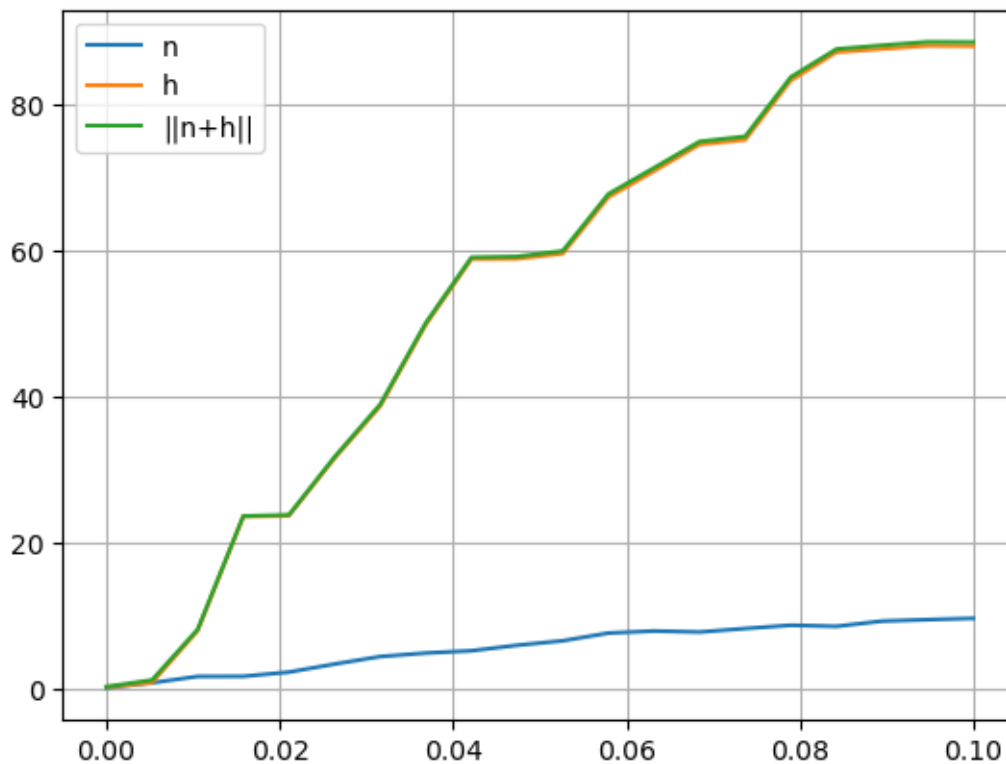
          errors_n = np.array(errors_n)
          errors_h = np.array(errors_h)

          mean_deviation.append([errors_n.mean(), errors_h.mean()])
          std_deviation.append([errors_n.std(), errors_h.std()])

      mean_deviation = np.array(mean_deviation)
```

```
[74]: plt.plot(
        noise_values,
        mean_deviation[:,0],
        label="n"
    )

    plt.plot(
        noise_values,
        mean_deviation[:,1],
        label="h"
    )
    plt.plot(noise_values, np.sqrt(mean_deviation[:,1]**2 +
        ↪ mean_deviation[:, 0]**2), label="||n+h||")
    plt.grid(True)
    plt.legend()
    plt.show()
```



Теперь можно видеть, что ошибка восстановления параметров имеет практически монотонную зависимость от уровня шума. Причем труднее всего восстанавливается параметр толщины слоя h - всего при 10-процентном уровне шума ошибка восстановления этого параметра превышает 80 процентов.

1.6. Восстановление показателя преломления при известной толщине слоя

Теперь допустим, что нам известна толщина исследуемого слоя заранее, и попробуем восстановить параметр n среды.

```
[ ]: h_true = 1.5
     n_trues = [1.32, 0.67, 2.0, 3.29]

     for n_true in n_trues:
         r_true = r_total(
             np.array([n1, n_true, n2]) * k0,
             h_true,
             theta0
         )

         n_vals = np.linspace(0, 15, 2000)

         R = [r_total(np.array([n1, n, n2]) * k0, h_true, theta0) for n in n_vals]

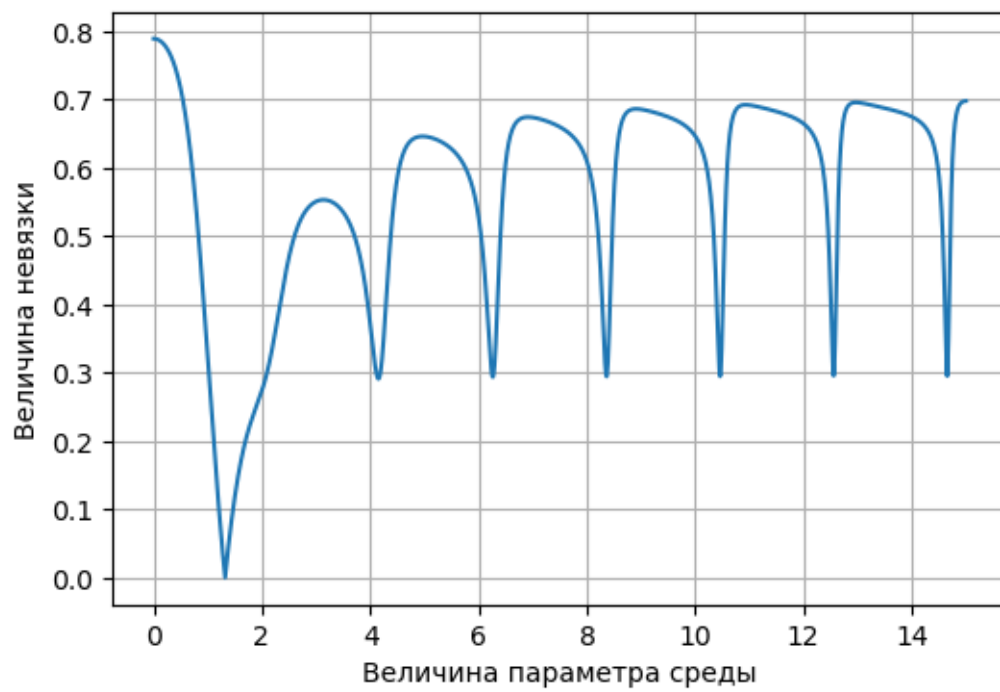
         loss = np.abs(R - r_true) # Невязка

         print(f"Минимальное значение: {n_vals[np.argmin(loss)]}")
         print(f"Истинное значение: {n_true}")

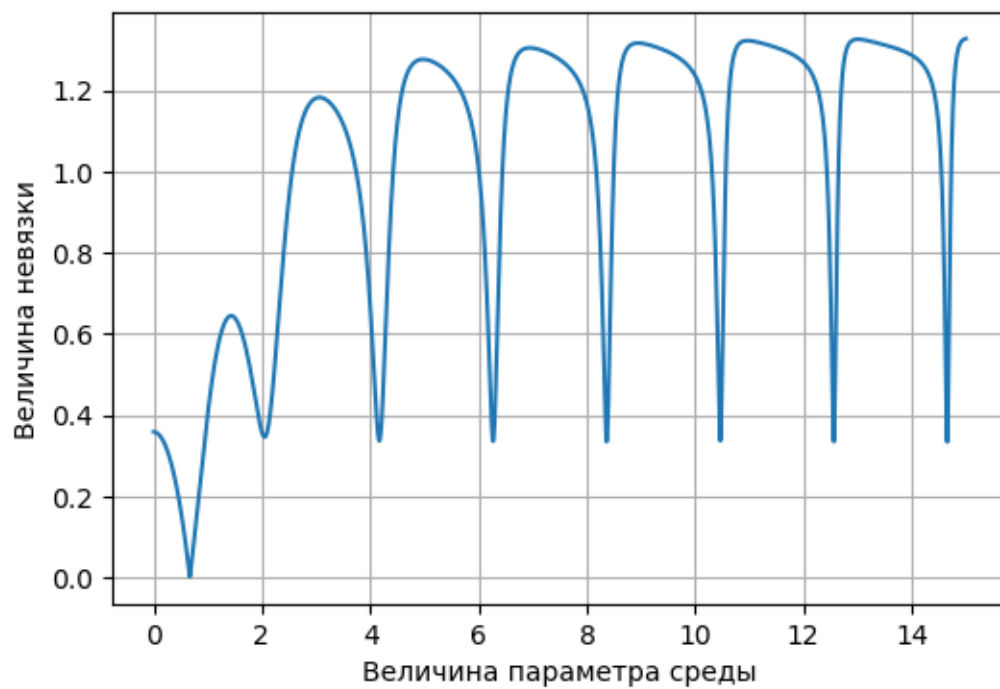
         plt.figure(figsize=(6, 4))
         plt.plot(n_vals, loss)
         plt.xlabel("Величина параметра среды")
         plt.ylabel("Величина невязки")
         plt.grid(True)
         plt.show()
```

Минимальное значение: 1.3206603301650826

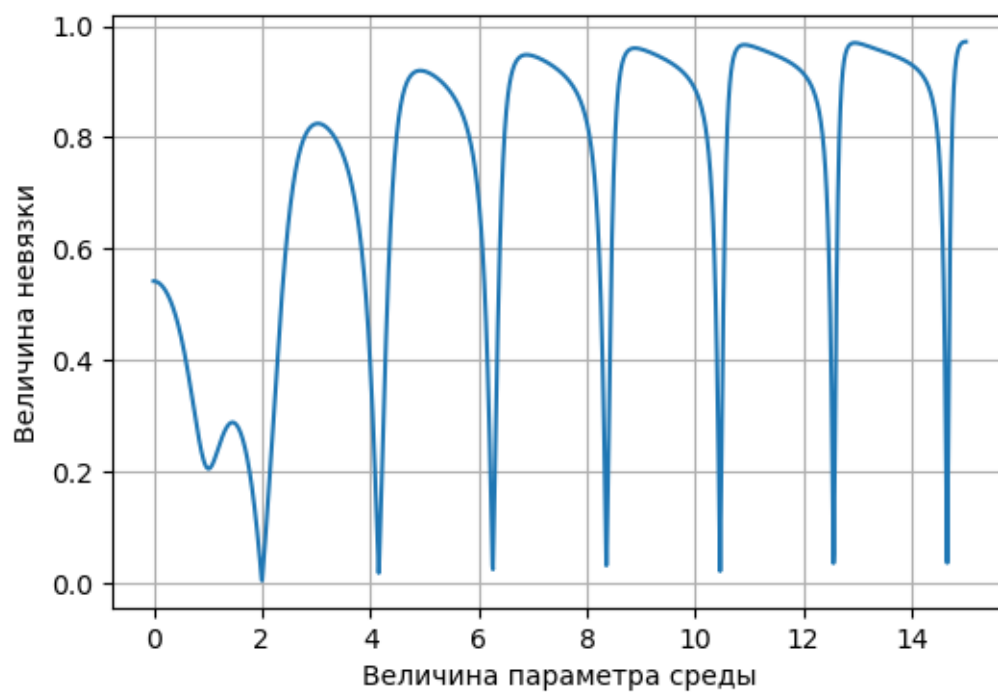
Истинное значение: 1.32



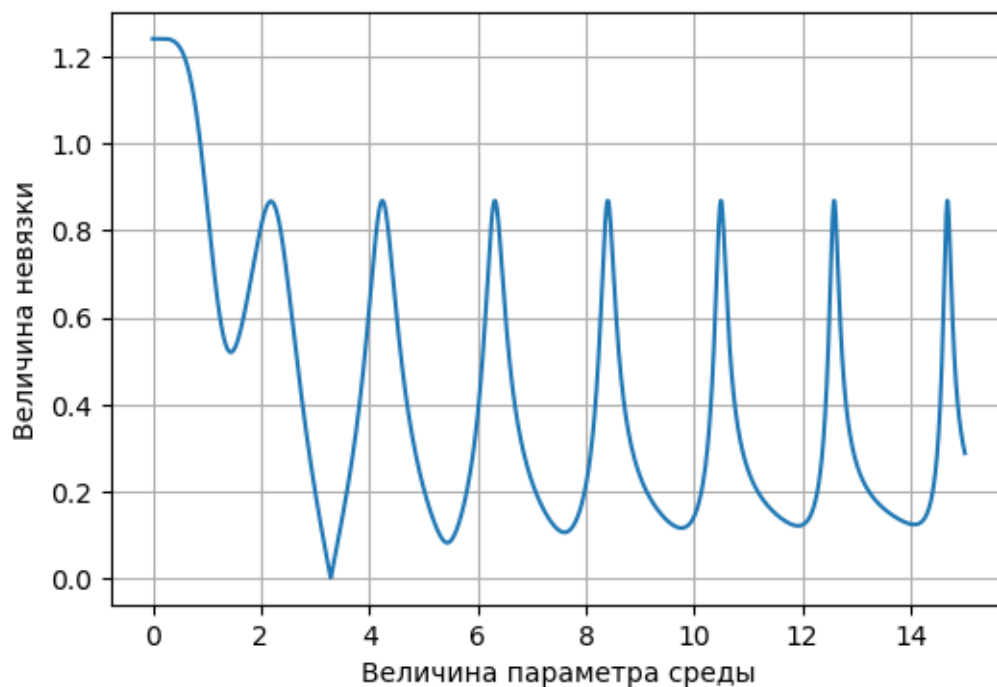
Минимальное значение: 0.6678339169584793
 Истинное значение: 0.67



Минимальное значение: 2.003501750875438
Истинное значение: 2.0



Минимальное значение: 3.2866433216608306
Истинное значение: 3.29



На графиках видно, что при известной толщине слоя функция невязки имеет хорошо выраженный минимум. Найденное значение показателя преломления практически совпадает с истинным, что говорит о значительно лучшей обусловленности задачи по сравнению со случаем одновременного восстановления двух параметров.

2. Численные методы решения обратной задачи

Примерный план:

- 1) Сформулировать обратную задачу
- 2) Выбрать численные методы
- 3) Для каждого метода:
 - Зафиксировать толщину слоя
 - Решить обратную задачу на незашумленных данных
 - Решить обратную задачу на данных с шумом и проверить устойчивость